

Hadoop HDFS

核心架构设计文档

基于 release-3.3.5-RC0 版本源码深度分析

项目	Apache Hadoop HDFS
模块	hadoop-hdfs-project / hadoop-hdfs
分支版本	release-3.3.5-RC0
源文件总数	~716 个 Java 文件
分析范围	NameNode / DataNode / BlockManager / HA / Snapshot / EC
文档生成	自动化源码分析工具

目录

- 一、HDFS 整体架构概览
- 二、模块源码结构分析
- 三、NameNode 核心架构
- 四、DataNode 核心架构
- 五、BlockManager 块管理
- 六、HA 高可用机制（状态模式）
- 七、文件系统树（INode 层次结构）
- 八、编辑日志与持久化机制
- 九、核心流程时序分析
- 十、设计模式总结
- 十一、关键场景调用链
- 十二、核心源文件清单

一、HDFS 整体架构概览

HDFS (Hadoop Distributed File System) 是 Apache Hadoop 的核心存储组件，采用主从架构 (Master/Slave)，由一个 NameNode 和多个 DataNode 组成。NameNode 负责管理文件系统命名空间（元数据），DataNode 负责实际的数据块存储。HDFS 采用一次写入、多次读取 (Write-Once-Read-Many) 的访问模型，适用于大规模数据集的批处理场景。

1.1 架构总览图

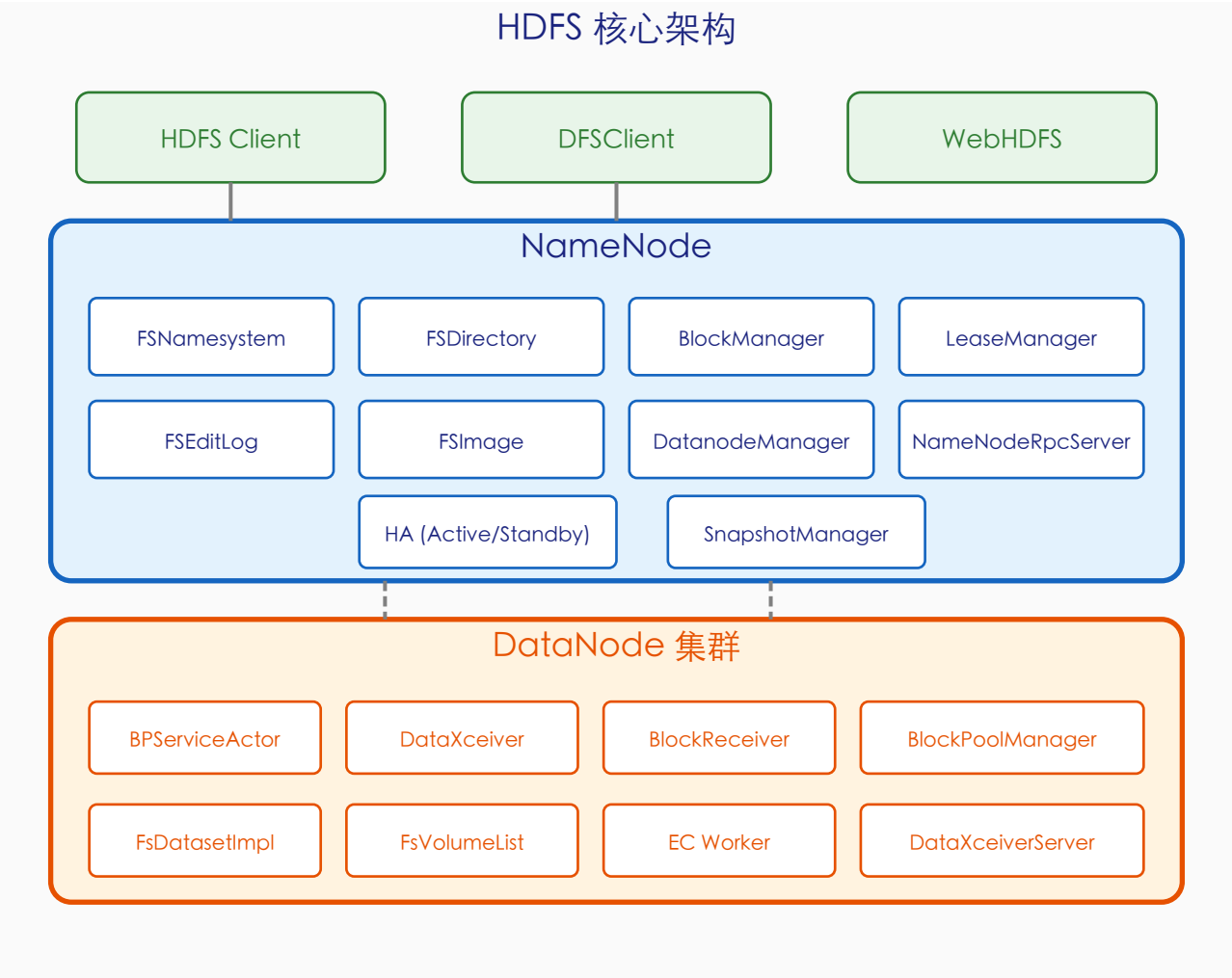


图 1：HDFS 核心架构全景图

1.2 核心组件职责

组件	职责	关键类
NameNode	管理文件系统命名空间，维护文件→块映射，协调数据块副本	NameNode, FSNamesystem, FSDirectory

组件	职责	关键类
DataNode	存储实际数据块，执行块创建/删除/复制，向NN发送心跳	DataNode, BPServiceActor, DataXceiver
BlockManager	管理块的副本状态、放置策略、块报告、块重建	BlockManager, DatanodeManager, BlockPlacementPolicy
FSEditLog	编辑日志持久化，记录所有命名空间变更操作	FSEditLog, EditLogOutputStream, JournalManager
FSImage	文件系统镜像（Checkpoint），周期性合并编辑日志	FSImage, FSImageFormat, SaveNamespace
HA 子系统	高可用状态机(Active/Standby/Observer)，自动故障转移	HAState, ActiveState, StandbyState, ZKFC

二、模块源码结构分析

hadoop-hdfs 模块包含约 716 个 Java 源文件，代码总量超过 30 万行。以下是各子包的文件分布和核心职责。

子包	文件数	核心职责	关键大文件
server/namenode/	~210	NameNode核心实现	FSNamesystem(341KB), NameNodeRpcServer(98KB)
server/datanode/	~131	DataNode核心实现	DataNode(165KB), DataXceiver(58KB)
server/blockmanagement/	68	块管理子系统	BlockManager(223KB), DatanodeManager(87KB)
server/protocol/	42	服务端协议定义	DatanodeProtocol, NamenodeProtocol
protocolPB/	24	ProtoBuf协议翻译层	ClientNamenodeProtocolPB 等
qjournal/	33	Quorum Journal HA	QuorumJournalManager(32KB)
server/namenode/ha/	-	HA状态机实现	HAState, ActiveState, StandbyState
server/namenode/snapshot/	-	快照管理	SnapshotManager(31KB), DirectorySnapshottableFeature
tools/	17+	管理工具	Balancer, Mover, DFSAdmin
util/	21	通用工具类	DFSUtil, EnumCounters

表 1：HDFS 模块子包分布一览

三、NameNode 核心架构

3.1 NameNode 启动流程

NameNode 是 HDFS 的核心元数据管理节点。其启动流程经过精心设计，确保文件系统命名空间的安全加载和服务的有序启动。入口方法为 `NameNode.main()` → `createNameNode()` → `new NameNode(conf)` → `initialize()`。

3.1.1 createNameNode() 方法 — 启动选项分发

`createNameNode()` 解析命令行参数，根据 `StartupOption` 分发到不同启动路径：

StartupOption	操作	说明
FORMAT	<code>format(conf, ...)</code>	格式化文件系统，初始化存储目录
GENCLUSTERID	<code>NNStorage.newClusterID()</code>	生成新的集群 ID
ROLLBACK	<code>doRollback(conf, true)</code>	回滚到升级前的版本
BOOTSTRAPSTANDBY	<code>BootstrapStandby.run()</code>	初始化 Standby NameNode
INITIALIZESHAREDEDITS	<code>initializeSharedEdits()</code>	初始化 QJM 共享编辑日志
BACKUP/CHECKPOINT	<code>new BackupNode()</code>	创建备份/检查点节点
default（正常启动）	<code>new NameNode(conf)</code>	创建 NameNode 实例，进入 <code>initialize()</code>

3.1.2 initialize() 方法 — 核心初始化序列

`initialize()` 是 NameNode 核心初始化流程，按以下严格顺序执行：

步骤	方法调用	说明
1	<code>UserGroupInformation.setConfiguration()</code>	配置安全认证框架（Kerberos等）
2	<code>loginAsNameNodeUser()</code>	以 NameNode 身份登录（Kerberos 认证）
3	<code>NameNode.initMetrics()</code>	初始化指标监控系统和 JVM 暂停监控器
4	<code>startHttpServer()</code>	启动 HTTP 服务（仅 NAMENODE 角色）
5	<code>loadNamesystem(conf)</code>	【关键】加载 FSImage + 重放 EditLog 恢复命名空间
6	<code>createRpcServer()</code>	【关键】创建 RPC 服务器（ClientProtocol 等）
7	<code>startCommonServices()</code>	【关键】启动 BlockManager、RPC监听、ServicePlugin

其中 `startCommonServices()` 内部调用 `namesystem.startCommonServices()` 启动 `BlockManager` 的块重建线程、心跳检测线程等，然后调用 `rpcServer.start()` 开始接受客户端请求。

3.2 FSNamesystem — 命名空间核心状态机

`FSNamesystem` 是 `NameNode` 的核心类（源码 341KB，约 9200+ 行），管理文件系统所有命名空间操作，是 HDFS 最大的单文件。它扮演“门面”角色（Facade Pattern），协调 `FSDirectory`、`BlockManager`、`FSEditLog`、`LeaseManager` 等子系统。

关键设计特征：

- 双重锁机制：先获取 `FSNamesystem` 写锁（`RwLockMode.FS`），再获取 `dir` 写锁
- 操作分类检查：通过 `checkOperation(OperationCategory.WRITE/READ)` 确保 HA 状态合法
- 安全模式：启动期间只允许读操作，等块报告达到阈值后自动退出
- 编辑日志同步：每次写操作后调用 `getEditLog().logSync()` 持久化

3.3 NameNodeRpcServer — RPC 服务层

`NameNodeRpcServer` (98KB) 实现了多种 `Protocol` 接口，是客户端和 `DataNode` 与 `NameNode` 通信的入口：

- `ClientProtocol`：客户端操作接口（`create/open/rename/delete/addBlock` 等）
- `DatanodeProtocol`：`DataNode` 通信接口（心跳、块报告、命令下发）
- `NamenodeProtocol`：`NameNode` 间通信接口（获取 `EditLog`、块报告等）
- `HAServiceProtocol`：HA 状态管理接口（状态切换、健康检查）

四、DataNode 核心架构

4.1 DataNode 组件结构

DataNode (165KB) 负责实际的块数据存储和传输。一个 DataNode 进程可同时服务多个 NameNode (Federation 模式)，通过 BlockPoolManager 管理多个块池。

组件	职责	关键机制
BPOfferService	管理一对NN（主备）的通信	封装两个 BPServiceActor
BPServiceActor	独立线程，执行心跳/块报告/IBR循环	offerService() 主循环
DataXceiverServer	监听数据传输端口，创建 DataXceiver	每个连接一个线程
DataXceiver	处理单个数据传输请求（读/写/复制）	Pipeline 写入、块校验
BlockReceiver	接收数据块并写入本地磁盘	校验和验证、Pipeline 转发
FsDatasetImpl	管理本地文件系统上的块存储	多卷管理、磁盘故障检测
BlockPoolManager	管理 DN 参与的所有块池	支持 Federation 多 NS

4.2 心跳与块报告机制 — BPServiceActor.offerService()

offerService() 是 DataNode 与 NameNode 通信的核心循环，运行在独立线程中。每轮循环按优先级执行以下操作：

核心循环流程：

1. 发送心跳（默认 3 秒）→ 返回 HeartbeatResponse (HA 状态、命令列表、块报告租约)
2. 更新 HA 状态 → 如 STANDBY→ACTIVE，确保在处理命令前完成状态切换
3. 处理 NN 命令 → KeyUpdateCommand 优先入队 (enqueueFirst)，其他命令普通入队
4. 发送增量块报告 (IBR) → 报告自上次以来的块变更
5. 发送全量块报告（默认 6 小时）→ 使用租约机制防止并发
6. 发送缓存报告 → 报告 DN 缓存的数据块状态
7. 等待下一轮 → ibrManager.waitTillNextIBR(heartbeatWaitTime)

4.3 DataXceiver.writeBlock() — Pipeline 写入

writeBlock() 实现了 HDFS 数据写入 Pipeline 的核心逻辑，流程如下：

1. 确定角色 (isClient / isDatanode / isTransfer) 和权限检查
2. 创建 BlockReceiver 打开本地写入器

3. 与下游 DataNode 建立 Socket 连接（含 SASL 安全握手）
4. 转发 writeBlock 请求到下游节点
5. 读取下游连接应答，向上游发送应答
6. 通过 `blockReceiver.receiveBlock()` 接收数据并转发
7. 收尾处理（更新 `GenerationStamp`、关闭块、更新指标）

五、BlockManager 块管理

5.1 BlockManager 概览

BlockManager（223KB，约 5800+ 行）是 NameNode 中最大的组件之一，负责管理所有数据块的生命周期。核心职责包括：

- 块→DataNode 映射管理（blocksMap）
- 块副本放置策略（BlockPlacementPolicy）
- 块报告处理（processReport）
- 块重建（ReplicationMonitor / RedundancyMonitor）
- 块失效管理（InvalidateBlocks）
- 心跳管理（HeartbeatManager）

5.2 副本放置策略 — BlockPlacementPolicyDefault

chooseTargetInOrder() 实现了经典的三副本放置算法，这是 HDFS 高可用性和性能的关键设计：

副本	放置位置	选择方法	设计考量
第1副本	本地节点（Writer 所在）	chooseLocalStorage()	减少网络传输
第2副本	远程机架上的一个节点	chooseRemoteRack()	容忍机架故障
第3副本	第2副本同机架的不同节点	chooseLocalRack(dn1)	平衡可用性与带宽
更多副本	随机选择	chooseRandom()	负载均衡

容错重试机制：当节点不足时，第一次重试允许选择陈旧节点（avoidStaleNodes=false），第二次重试放宽存储类型约束。最终节点通过 getPipeline() 排序形成写入管道。

5.3 块报告处理 — processReport()

processReport() 处理 DataNode 发送的全量块报告，核心采用差异化处理策略：

首次报告（processFirstBlockReport）：高效处理，用于加速 NameNode 重启

后续报告（processReport 内部）：通过 reportDiff() 计算五个差异队列：

队列	含义	处理方法
toAdd	新增的块（DN有，NN无）	addStoredBlock()
toRemove	需移除的块	removeStoredBlock()
toInvalidate	需失效的块（NN有，DN不应有）	addToInvalidates()

队列	含义	处理方法
toCorrupt	损坏的块	markBlockAsCorrupt()
toUC	正在构建的块 (Under Construction)	addStoredBlockUnderConstruction()

注意：processReport()
持有全局写锁（RwLockMode.GLOBAL），在安全模式下会丢弃非首次块报告以减少启动时间。

六、HA 高可用机制（状态模式）

6.1 状态模式实现

HDFS HA 采用经典的状态模式（State Pattern）实现，由抽象基类 `HAState` 和三个具体状态类组成：

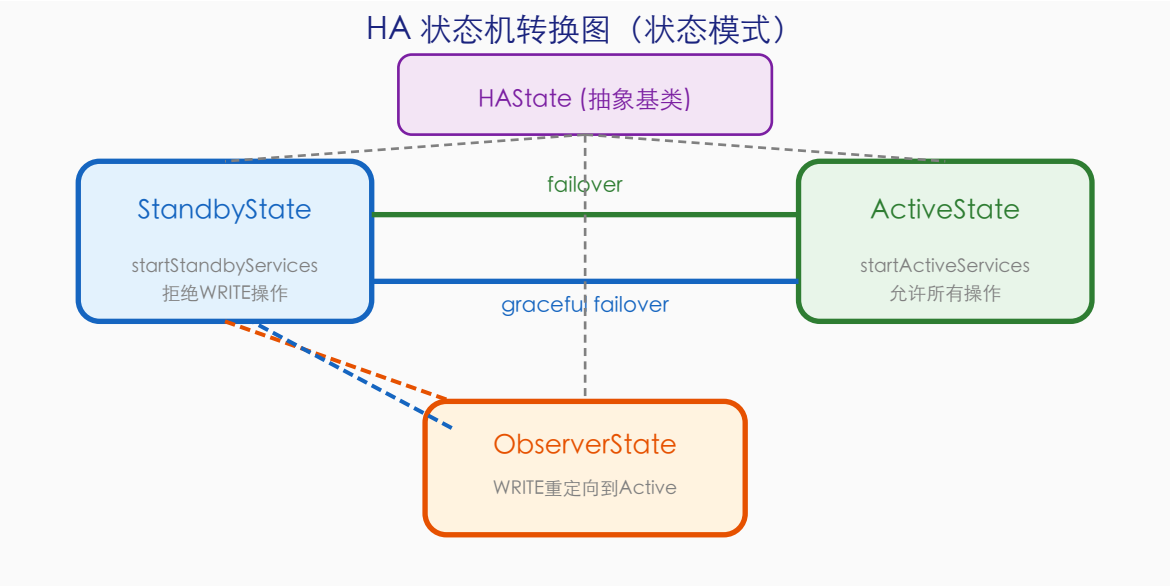


图 2：HA 状态机转换图

6.2 HAState 核心方法 — `setStateInternal()`

`setStateInternal()` 是状态转换的模板方法（Template Method Pattern），按固定步骤执行：

步骤	方法	说明	持锁状态
①	<code>prepareToExitState()</code>	准备退出当前状态（如取消 checkpoint）	无锁
②	<code>s.prepareToEnterState()</code>	准备进入新状态（前置条件检查）	无锁
③	<code>context.writeLock()</code>	获取写锁	加锁
④	<code>exitState()</code>	退出当前状态（停止服务）	持锁
⑤	<code>context.setState(s)</code>	设置新状态	持锁
⑥	<code>s.enterState()</code>	进入新状态（启动服务）	持锁
⑦	<code>updateLastHATransitionTime()</code>	记录转换时间戳	持锁

6.3 三种状态的行为差异

特性	ActiveState	StandbyState	ObserverState
允许的操作	READ + WRITE	UNCHECKED + Stale READ	READ（重定向WRITE到Active）
填充副本队列	是	否	否
可转换到	Standby	Active / Observer	Standby
enterState()	startActiveServices()	startStandbyServices()	startStandbyServices()
exitState()	stopActiveServices()	stopStandbyServices()	stopStandbyServices()

6.4 Quorum Journal Manager (QJM)

QuorumJournalManager（32KB）是 HA 共享编辑日志的核心实现，通过写入远程 JournalNode 集群实现编辑日志的高可用：

- Active NameNode 将编辑日志写入多数 JournalNode（多数派确认）
- Standby NameNode 通过 EditLogTailer 定期拉取编辑日志
- StandbyCheckpointter 周期性合并 FSImage + EditLog，减少重启时间
- Fencing 机制确保同一时刻只有一个 Active NameNode 可写入

七、文件系统树（INode 层次结构）

7.1 INode 类继承体系

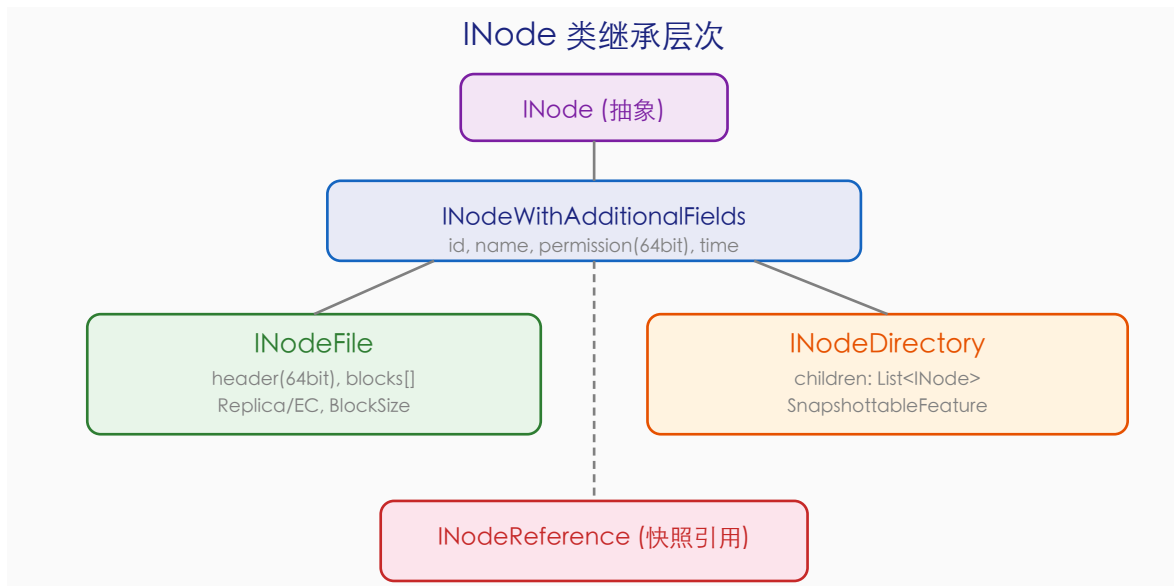


图 3：INode 类继承层次图

HDFS 文件系统树由 INode 层次结构在内存中维护，所有文件和目录操作都通过 FSDirectory 在此树上执行：

- INode：抽象基类，定义文件/目录的通用接口
- INodeWithAdditionalFields：添加 id、name、permission（64位编码 user+group+mode）、时间戳
- INodeFile：文件节点，header 字段使用 64 位编码存储 StoragePolicy(4bit) + 副本/EC策略(12bit) + blockSize(48bit)
- INodeDirectory：目录节点，children 列表默认初始容量仅为 2（性能分析的经验值）
- INodeReference：快照引用节点，支持快照中的文件/目录引用

7.2 快照感知操作

INodeDirectory

的所有子节点操作（getChild/addChild/removeChild/getChildrenList）都是“快照感知”的：

- 当 snapshotId == CURRENT_STATE_ID 时，操作当前视图（getCurrentChildrenList）
- 当访问快照视图时，通过 DirectoryWithSnapshotFeature 的 DiffList 查找历史状态
- 修改操作（addChild/removeChild）在快照存在时会记录 diff，实现写时复制（Copy-on-Write）语义
- SnapshotManager 全局管理快照 ID 计数器和数量限制

八、编辑日志与持久化机制

8.1 FSEditLog 双缓冲区同步机制

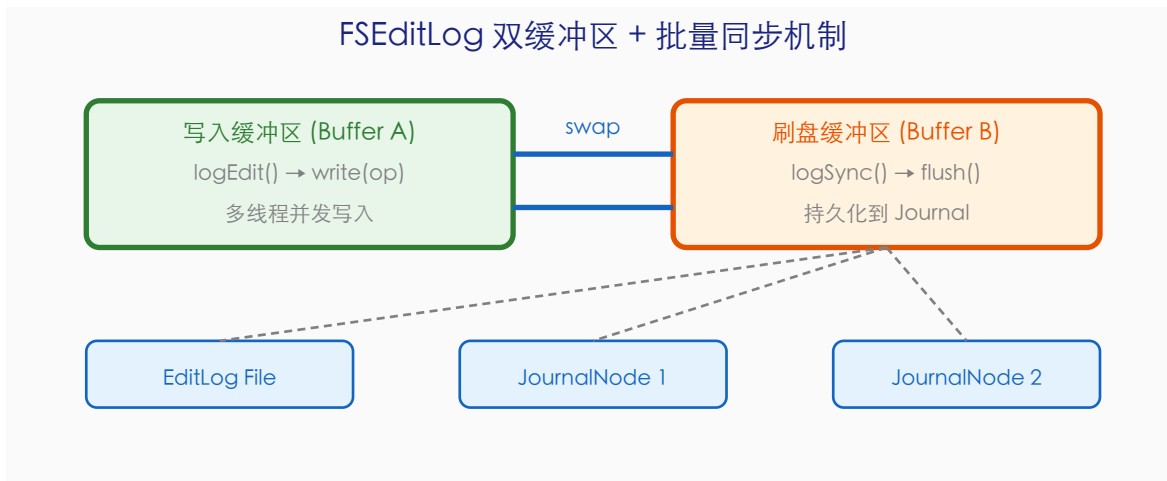


图 4：FSEditLog 双缓冲区同步机制

8.2 logEdit() — 编辑日志写入

logEdit() 是编辑日志写入的入口方法，关键流程：

1. synchronized 块内：waitIfAutoSyncScheduled() → beginTransaction(op) → doEditTransaction(op)
2. doEditTransaction() 将操作写入缓冲区（editLogStream.write(op)），检查是否需要强制同步
3. 如需同步，在 synchronized 块外调用 logSync()，允许其他线程继续写入

8.3 logSync() — 批量同步与双缓冲区交换

logSync() 是编辑日志持久化的核心，实现了高效的批量同步机制：

阶段	操作	持锁状态	说明
等待	while (mytxid > synctxid && isSyncRunning) wait()	synchronized	等待其他线程同步完成
检查	if (mytxid <= synctxid) return	synchronized	事务已被批量同步
交换	editLogStream.setReadyToFlush()	synchronized	交换双缓冲区
释放锁	doneWithAutoSyncScheduling()	synchronized	通知等待的写线程
刷盘	logStream.flush()	无锁	实际持久化（耗时操作）

阶段	操作	持锁状态	说明
更新	synctxid = lastJournalledTxId; notifyAll()	synchroniz ed	更新已同步ID并唤醒

- 关键设计要点：
- 双缓冲区机制让写入和刷盘可以并行进行，极大提升吞吐量
 - 批量同步机制使多个线程的编辑被一次 flush() 一起持久化
 - 同步失败时 NameNode 立即调用 terminate(1) 退出（Fail-Fast 策略），防止数据不一致

8.4 FSImage — 文件系统镜像

- FSImage（60KB）负责文件系统镜像的持久化，核心机制：
- Checkpoint：将 FSImage + EditLog 合并为新的 FSImage
 - SaveNamespace：将内存中的命名空间序列化到磁盘
 - 加载流程：启动时先加载最新的 FSImage，再重放后续的 EditLog 事务
 - Standby Checkpointer：Standby NN 定期执行 Checkpoint 并上传到 Active NN

九、核心流程时序分析

9.1 文件写入完整流程

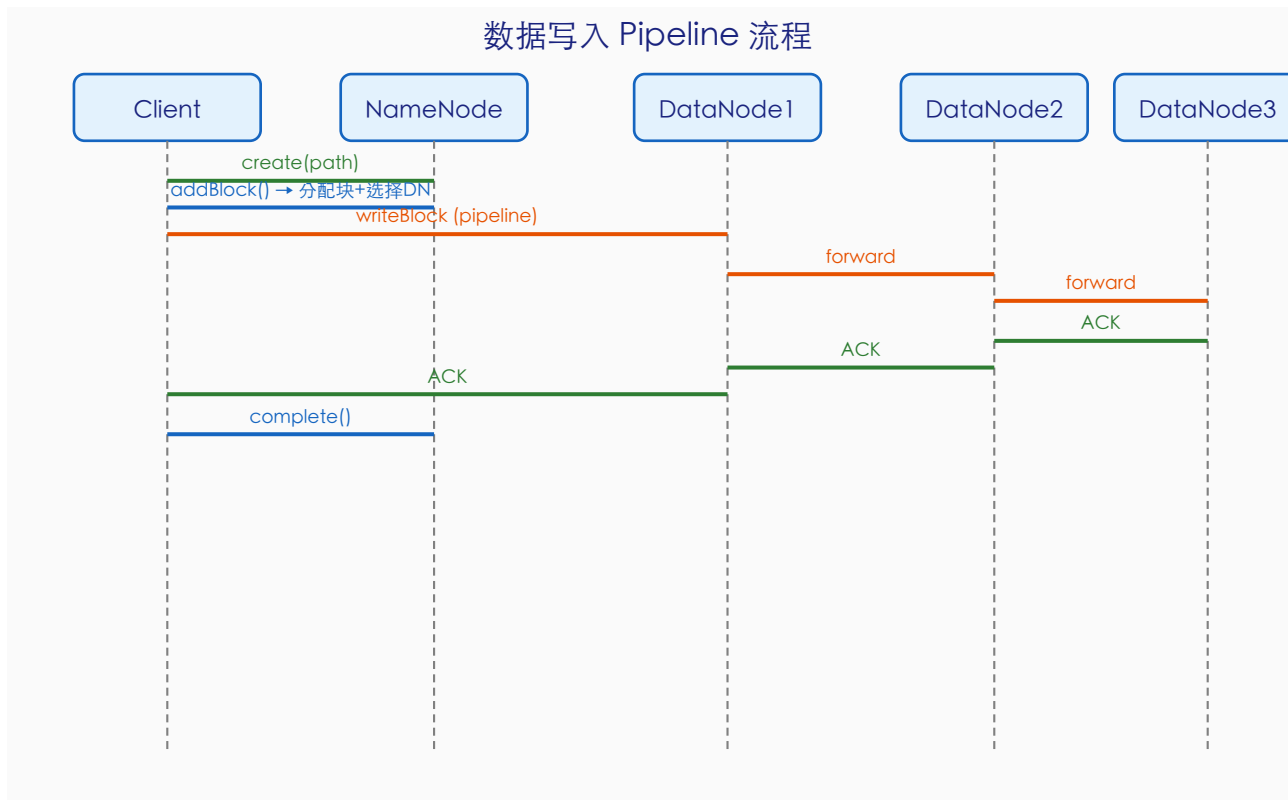


图 5：数据写入 Pipeline 时序图

文件写入详细流程（`startFileInt` 方法分析）：

1. 路径校验：`DFSUtil.isValidName(src)` + 保留名检查
2. 副本策略决策：`SHOULD_REPLICATE` → `verifyReplication()` / 否则检查 EC 策略
3. 获取写锁 `writeLock(RwLockMode.FS)`
4. `checkOperation(WRITE)` + `checkNameNodeSafeMode()` 确保 Active 且非安全模式
5. `FSDirWriteFileOp.resolvePathForStartFile()` 解析路径
6. 处理加密区域（释放锁→生成 EDEK→重新获取锁→重新解析路径）
7. `dir.writeLock()` → `FSDirWriteFileOp.startFile()` 创建文件 `INode`
8. 释放锁后 `getEditLog().logSync()` 持久化编辑日志

9.2 心跳处理流程

DataNode 心跳主循环（`BPSERVICEActor.offerService`）：

每轮循环完整流程：

- ① 判断心跳是否到期 → ② 发送心跳并获取响应 → ③ 更新 HA 状态（先于命令处理）
- ④ 处理 NN 下发命令（KeyUpdate 优先入队）→ ⑤ 发送 IBR 增量块报告
- ⑥ 发送全量块报告（使用租约）→ ⑦ 发送缓存报告 → ⑧ 等待下一轮

NameNode 命令类型 (DatanodeProtocol.DNA_* 常量) :

- DNA_TRANSFER = 1 : 传输块到其他 DataNode
- DNA_INVALIDATE = 2 : 删除指定块
- DNA_SHUTDOWN = 3 : 关闭 DataNode
- DNA_REGISTER = 4 : 要求重新注册
- DNA_FINALIZE = 5 : 最终化升级
- DNA_RECOVERBLOCK = 6 : 块恢复
- DNA_BALANCERBANDWIDTHUPDATE = 10 : 更新均衡器带宽
- DNA_CACHE = 11 / DNA_UNCACHE = 12 : 缓存/取消缓存块
- DNA_ERASURE_CODING_RECONSTRUCTION = 13 : 纠删码重建

十、设计模式总结

设计模式	应用场景	关键类	设计意图
状态模式 (State)	HA 状态管理	HAState, ActiveState, StandbyState	将状态特定行为封装在独立类中，状态转换清晰可扩展
模板方法 (Template Method)	HA 状态转换	HAState.setStateInternal()	定义转换骨架 (prepareExit→prepareEnter→exit→enter)，子类实现细节
策略模式 (Strategy)	块副本放置	BlockPlacementPolicy, Default/Rack-Aware	算法可插拔，支持自定义副本放置策略
门面模式 (Facade)	NameNode 核心	FSNamesystem	统一入口协调 FSDirectory/Block Manager/FSEditLog/LeaseManager
观察者模式 (Observer)	编辑日志	JournalSet + 多个 JournalManager	一次写入操作广播到所有 Journal (File + QJM)
工厂模式 (Factory)	RPC 协议	ProtocolPB 翻译层	ProtoBuf 序列化与服务端接口的解耦
组合模式 (Composite)	文件系统树	Inode / InodeFile / InodeDirectory	统一的文件/目录树操作接口
装饰器模式 (Decorator)	Inode 特性	Feature 接口 (AclFeature, XAttrFeature等)	通过 Feature[] 动态扩展 Inode 功能
双缓冲区	编辑日志同步	FSEditLog logSync/setReadyToFlush	写入和刷盘并行，提升吞吐量
租约模式 (Lease)	文件写入控制	LeaseManager / BlockReportLeaseManager	通过租约控制并发写入和块报告

表 2：HDFS 核心设计模式一览

十一、关键场景调用链

11.1 文件创建调用链

```
Client.create() → DFSClient.create() → NameNodeRpcServer.create() →  
FSNamesystem.startFile() → startFileInt() →  
FSDirWriteFileOp.resolvePathForStartFile() → FSDirWriteFileOp.startFile() →  
FSEditLog.logOpenFile() → logSync()
```

11.2 块分配调用链

```
Client.addBlock() → NameNodeRpcServer.addBlock() →  
FSNamesystem.getAdditionalBlock() → BlockManager.chooseTarget4NewBlock() →  
BlockPlacementPolicyDefault.chooseTarget() → chooseTargetInOrder() →  
chooseLocalStorage() / chooseRemoteRack() / chooseLocalRack()
```

11.3 心跳处理调用链

```
BPServiceActor.offerService() → sendHeartBeat() →  
DatanodeProtocol.sendHeartbeat() → NameNodeRpcServer.sendHeartbeat() →  
FSNamesystem.handleHeartbeat() → BlockManager.DatanodeManager.handleHeartbeat()  
→ HeartbeatManager.updateHeartbeat()
```

11.4 块报告处理调用链

```
BPServiceActor.blockReport() → DatanodeProtocol.blockReport() →  
NameNodeRpcServer.blockReport() → BlockManager.processReport() →  
processFirstBlockReport() ■ processReport(■■) → reportDiff() → addStoredBlock()  
/ removeStoredBlock()
```

11.5 HA Failover 调用链

```
ZKFC.fenceOldActive() / becomeActive() → HAServiceProtocol.transitionToActive()  
→ NameNode.transitionToActive() → HASTate.setState(ACTIVE_STATE) →  
ActiveState.setStateInternal() → StandbyState.prepareToExitState() →  
ActiveState.prepareToEnterState() → StandbyState.exitState() →  
stopStandbyServices() → ActiveState.enterState() → startActiveServices()
```

11.6 快照创建调用链

```
Client.createSnapshot() → NameNodeRpcServer.createSnapshot() →  
FSNamesystem.createSnapshot() → SnapshotManager.createSnapshot() →  
INodeDirectory.addSnapshot() → DirectorySnapshottableFeature.addSnapshot() → ■■  
snapshotCounter / numSnapshots
```

11.7 Pipeline 数据写入调用链

```
Client.DFSOutputStream.writeChunk() → DataStreamer.run() →  
createBlockOutputStream() → DataXceiver.writeBlock() → getBlockReceiver() → ■■  
BlockReceiver → ■■■■■■ DN ■ Socket ■■ → new Sender(mirrorOut).writeBlock() →  
■■■■■■ → blockReceiver.receiveBlock(mirrorOut, mirrorIn, replyOut)
```

十二、核心源文件清单

源文件	大小	核心职责
NameNode.java	110KB	HDFS 主入口点，HA 状态管理
FSNamesystem.java	341KB	命名空间核心状态机（模块最大文件）
FSDirectory.java	75KB	内存目录树管理（INode 层次）
FSEditLog.java	64KB	编辑日志管理（双缓冲区同步）
FSImage.java	60KB	文件系统镜像持久化
NameNodeRpcServer.java	98KB	NameNode RPC 服务端
DataNode.java	165KB	DataNode 核心实现
BPServiceActor.java	53KB	DataNode 心跳/块报告线程
DataXceiver.java	58KB	数据传输处理器（Pipeline 实现）
BlockReceiver.java	67KB	块数据接收与写盘
BlockManager.java	223KB	块管理核心（第二大文件）
BlockPlacementPolicyDefault.java	56KB	默认副本放置策略
DatanodeManager.java	87KB	DataNode 管理器
HeartbeatManager.java	19KB	心跳管理与过期检测
HAState.java	5KB	HA 状态机基类（状态模式）
ActiveState.java	2.4KB	Active 状态实现
StandbyState.java	4KB	Standby/Observer 状态实现
QuorumJournalManager.java	32KB	QJM 共享编辑日志客户端
INode.java	39KB	文件/目录抽象基类
INodeFile.java	42KB	文件 INode（64位 header 编码）
INodeDirectory.java	35KB	目录 INode（快照感知操作）
LeaseManager.java	22KB	文件租约管理
SnapshotManager.java	31KB	快照管理器
DatanodeProtocol.java	9KB	DN↔NN 通信协议接口

表 3：HDFS 核心源文件清单（24 个关键文件）